

ANALISE SINTÁTICA

Descrição do trabalho:

Nesta etapa, você deverá implementar um analisador léxico para a linguagem MicroPascal (μ -Pascal) cuja descrição fora mostrada em sala e disponibilizada para download, aqui transcrita:

PROGRAMA E BLOCO

$\langle \text{programa} \rangle ::= \text{program } \langle \text{identificador} \rangle ; \langle \text{bloco} \rangle .$
 $\langle \text{bloco} \rangle ::= \langle \text{parte de declarações de variáveis} \rangle \langle \text{comando composto} \rangle$

DECLARAÇÕES

$\langle \text{parte de declarações de variáveis} \rangle ::= \{ \text{var } \langle \text{declaração de variáveis} \rangle \{ ; \langle \text{declaração de variáveis} \rangle \} ; \}$
 $\langle \text{declaração de variáveis} \rangle ::= \langle \text{lista de identificadores} \rangle : \langle \text{tipo} \rangle$
 $\langle \text{lista de identificadores} \rangle ::= \langle \text{identificador} \rangle \{ , \langle \text{identificador} \rangle \}$
 $\langle \text{tipo} \rangle ::= \text{integer} \mid \text{real}$

COMANDOS

$\langle \text{comando composto} \rangle ::= \text{begin } \langle \text{comando} \rangle ; \{ \langle \text{comando} \rangle ; \} \text{end}$
 $\langle \text{comando} \rangle ::= \langle \text{atribuição} \rangle \mid \langle \text{comando composto} \rangle \mid \langle \text{comando condicional} \rangle \mid \langle \text{comando repetitivo} \rangle$
 $\langle \text{atribuição} \rangle ::= \langle \text{variável} \rangle := \langle \text{expressão} \rangle$
 $\langle \text{comando condicional} \rangle ::= \text{if } \langle \text{expressão} \rangle \text{ then } \langle \text{comando} \rangle [\text{else } \langle \text{comando} \rangle]$
 $\langle \text{comando repetitivo} \rangle ::= \text{while } \langle \text{expressão} \rangle \text{ do } \langle \text{comando} \rangle$

EXPRESSÕES

$\langle \text{expressão} \rangle ::= \langle \text{expressão simples} \rangle [\langle \text{relação} \rangle \langle \text{expressão simples} \rangle]$
 $\langle \text{relação} \rangle ::= = \mid < \mid <= \mid >= \mid >$
 $\langle \text{expressão simples} \rangle ::= [+ \mid -] \langle \text{termo} \rangle \{ (+ \mid -) \langle \text{termo} \rangle \}$
 $\langle \text{termo} \rangle ::= \langle \text{fator} \rangle \{ (* \mid /) \langle \text{fator} \rangle \}$
 $\langle \text{fator} \rangle ::= \langle \text{variável} \rangle \mid \langle \text{número} \rangle \mid (\langle \text{expressão} \rangle)$
 $\langle \text{variável} \rangle ::= \langle \text{identificador} \rangle$

Instruções:

Implemente um procedimento (ou função) CasaToken que deverá receber como parâmetro o token esperado pela gramática e compará-lo com o token corrente (registro montado pelo analisador léxico). Se estes forem iguais, o próximo token do programa será lido (chamada ao analisador léxico), caso contrário, deverá ser emitida uma mensagem com o erro encontrado e a linha do programa onde este ocorreu, encerrando o processo de compilação.

Implemente um procedimento (ou função) para cada símbolo não-terminal da gramática. Em cada procedimento (ou função), a escolha da regra de produção a ser utilizada depende do token lido pelo analisador léxico. Cada token do lado direito da regra deverá ser verificado pelo procedimento (ou função) CasaToken. Símbolos não-terminais do lado direito da regra correspondem a chamadas ao procedimento correspondente.

O corpo principal do compilador deverá ler o primeiro token do programa-fonte (chamada ao analisador léxico) e em seguida chamar o procedimento correspondente ao símbolo inicial da gramática. Ademais, teste o analisador sintático com exemplos de programas-fontes corretos e exemplos de erros sintáticos.

As mensagens devem ter os seguintes formatos (onde nn é o número da linha onde o erro foi detectado e lex é o lexema encontrado):

- nn:token nao esperado [lex].
- nn:fim de arquivo não esperado.

O que entregar?

Você deverá entregar nesta etapa:

1. Todos os arquivos fonte;
2. Relatório técnico contendo explicações do proposito de todas as structs e funções, assim como testes realizados com programas corretos e errados (no mínimo, 3 certos e 3 errados). Os programas testes deverão ser definidos de acordo com a gramática do MicroPascal.
3. Os resultados deverão apresentar, como saída do Analisador Sintático para cada código-fonte a ele submetido, a sequência de regras de produção que seriam usadas para analisar esse código-fonte, a partir do não terminal “programa” ou a árvore sintática a partir de função incorporada no código-fonte em C.

Regras:

- O trabalho poderá deverá ser desenvolvido em grupos de até 5 alunos;
- Não é permitido o uso de ferramentas para geração do analisador léxico;
- A implementação deverá ser realizada em C;
- Trabalhos total ou parcialmente iguais receberão avaliação nula;
- Ultrapassados cinco (5) dias da data definida para entrega, nenhum trabalho será recebido.